Code & Results

Aditya Katre

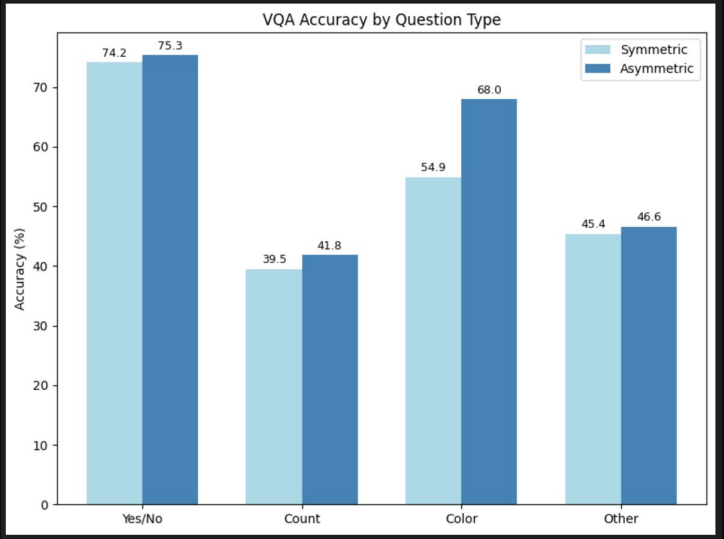
Comparison to a Baseline

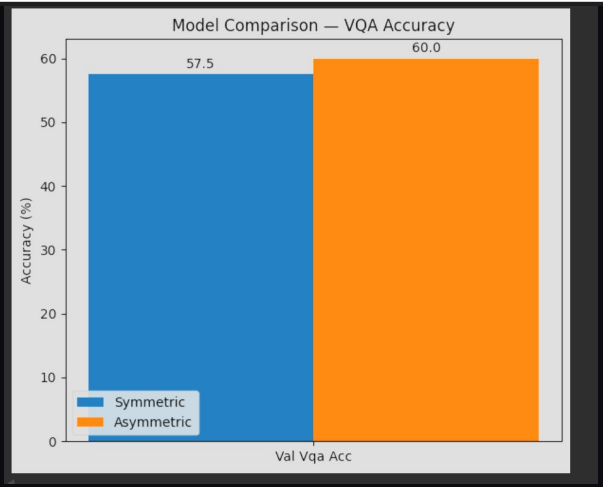
Table 2. Comparison results on VQA-E-Syn regarding answer accuracy. OA and AA represent overall and average accuracy, respectively.

Method	Number	Yes/No	Other	OA	AA
MUTAN [5]	49.23	65.52	60.27	61.07	58.34
BUTD [2]	52.68	64.07	65.72	64.13	60.83
MCAN [48]	59.19	71.70	72.37	71.09	67.76
LXMERT [41]	59.93	72.34	73.04	71.76	68.44
UNITER [9]	61.58	73.81	72.66	72.14	69.35
BLIP [20]	63.68	72.43	74.83	73.16	70.31
CMQEF [24]	60.02	71.63	72.20	71.03	67.95
VQA-E [23]	54.26	63.98	66.45	64.67	61.57
DMRFNet [50]	59.08	72.28	73.84	72.15	68.40
REX [8]	59.73	72.91	74.18	72.84	68.94
OFA-base [45]	60.48	73.26	75.37	73.49	69.70
VCIN [47]	61.59	73.47	73.10	72.43	69.39
MRVQA-A (ours)	57.02	72.80	73.41	71.89	67.75
MRVQA-C (ours)	63.20	76.03	75.72	74.81	71.65
MRVQA-E (ours)	**65.11**	**76.40**	**75.87**	**75.01**	**72.46**

https://arxiv.org/abs/2402.03896 - Page 7

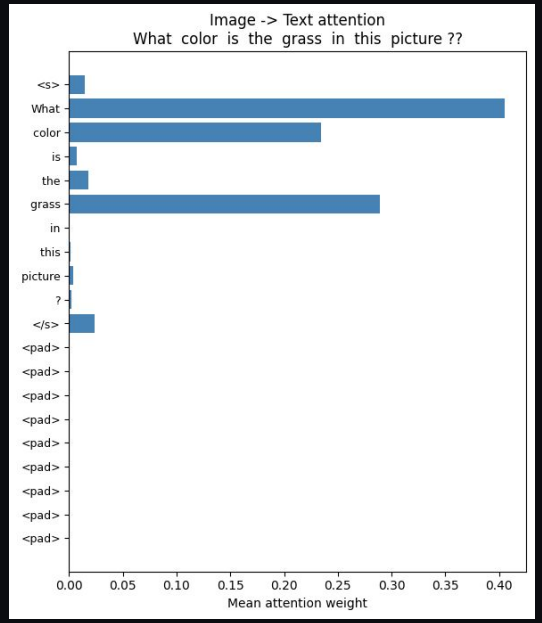
Multimodal Rationales for Explainable Visual Question Answering (2024)



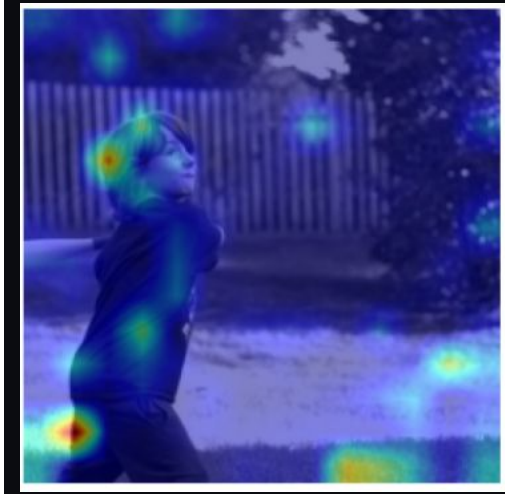


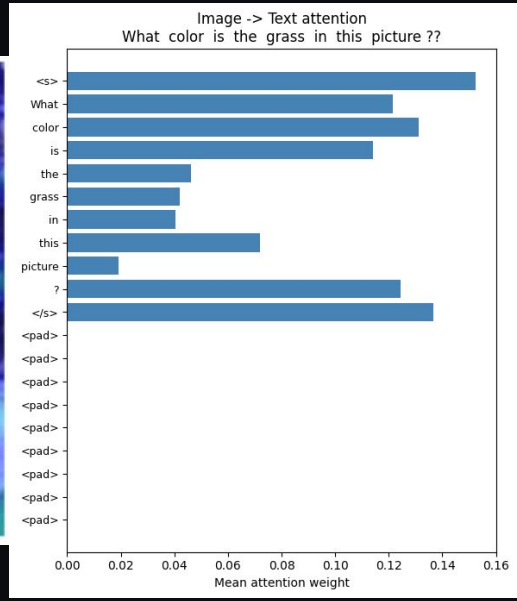
Our implementation

Comparison: What color is the grass in this picture?



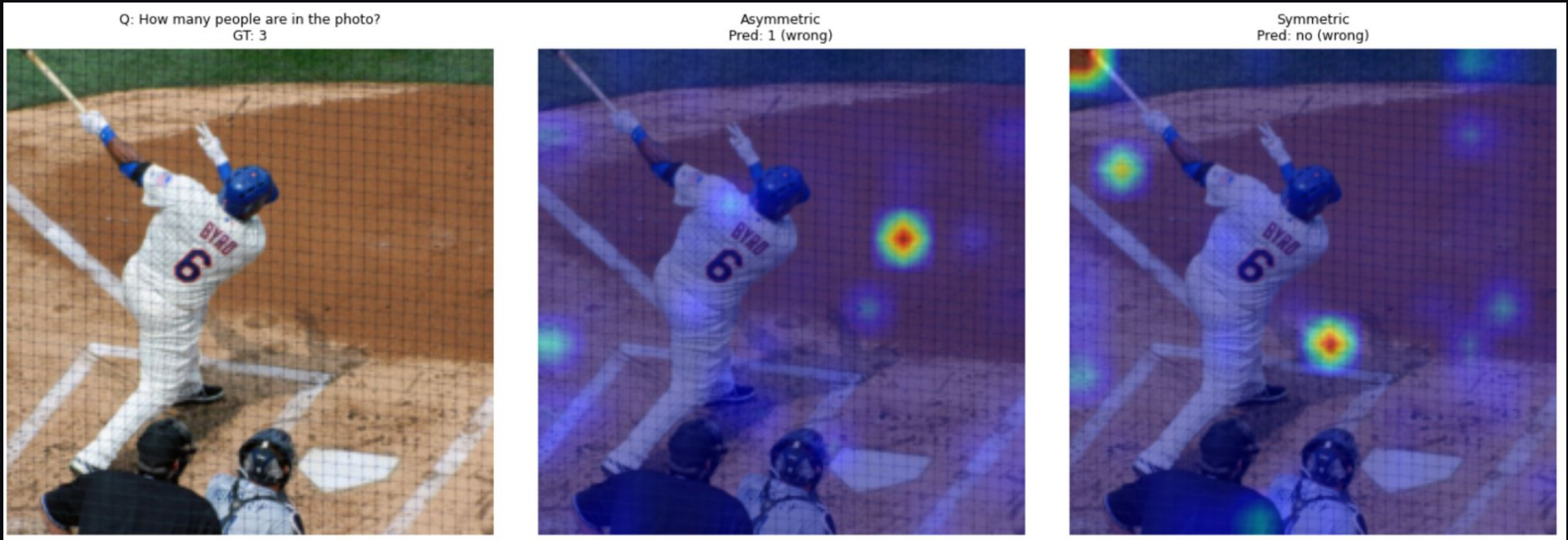
Asymmetric





Symmetric

Comparison: How many people are in the photo?



Code Snippets

Hyperparameters

```
NUM_ANSWERS       = 1000
EMBED_DIM         = 512
NUM_HEADS         = 8
DROPOUT           = 0.2
FREEZE_ENCODERS   = True
MAX_QUESTION_LEN  = 20
BATCH_SIZE        = 128
LEARNING_RATE     = 1e-4
WEIGHT_DECAY      = 1e-5
EPOCHS            = 18
SEED              = 42
USE_AMP           = True
```

- NUM_ANSWERS = 1000
  - Uses the top-1000 most frequent answers
- DROPOUT = .2
- BATCH_SIZE = 128
  - 128 examples per step
- set_seed(42)
  - will change later to make CI
- NUM_HEADS = 8
  - number of attention heads
- WEIGHT_DECAY = .00001
  - prevents overfitting

Preprocessing

```python
def build_answer_vocab(annotations_file, top_k=1000):
    """Build answer vocabulary from the top_k most frequent answers."""
    with open(annotations_file) as f:
        annotations = json.load(f)["annotations"]

    counter = Counter()
    for ann in annotations:
        counter[ann["multiple_choice_answer"]] += 1

    most_common = [ans for ans, _ in counter.most_common(top_k)]
    answer_to_idx = {ans: idx for idx, ans in enumerate(most_common)}
    idx_to_answer = {idx: ans for ans, idx in answer_to_idx.items()}
    return answer_to_idx, idx_to_answer


def get_image_transform(split="train"):
    """ImageNet-normalised transform for ViT-B/16 (224x224)."""
    if split == "train":
        return transforms.Compose([
            transforms.Resize(256),
            transforms.RandomCrop(224),
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
        ])
    return transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])

def get_h5_preprocess_transform():
    """Resize to 256x256 for HDF5 storage (no normalization, no crop randomness)."""
    return transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(256),
    ])
```

- `build_answer_vocab`
  - counts all answers across training set and keeps the 1,000 most common
  - maps each answer string to an integer index
- `get_image_transform`
  - applies random cropping and flipping to training data
  - applies center cropping for validation data
  - resizes to 224x224 for ViT
- normalization
  - uses mean and stdev from the ImageNet dataset, which was used to train ViT
  - ensures that colors do not causes issues during training

Compile Data into HDF5

```python
# --- HDF5 Preprocessing: convert raw JPEGs into a single contiguous file ---
h5_path = DATA_DIR / "vqa_images.h5"

if not h5_path.exists():
    preprocess = get_h5_preprocess_transform()
    image_dirs = [DATA_DIR / "images" / "train2014", DATA_DIR / "images" / "val2014"]

    # Collect all image paths
    all_paths = []
    for d in image_dirs:
        if d.exists():
            all_paths.extend(sorted(d.glob("*.jpg")))
    print(f"Found {len(all_paths):,} images to preprocess")

    with h5py.File(h5_path, "w") as h5f:
        imgs_ds = h5f.create_dataset(
            "images", shape=(len(all_paths), 256, 256, 3),
            dtype=np.uint8, chunks=(1, 256, 256, 3))
        ids_ds = h5f.create_dataset(
            "image_ids", shape=(len(all_paths),), dtype=np.int64)

        for i, path in enumerate(tqdm(all_paths, desc="Preprocessing images")):
            image_id = int(path.stem.split("_")[-1])
            img = Image.open(path).convert("RGB")
            img = preprocess(img)
            imgs_ds[i] = np.array(img)
            ids_ds[i] = image_id

    print(f"Saved {len(all_paths):,} images to {h5_path}")
else:
    print(f"HDF5 file already exists: {h5_path}")
```

- `build_answer_vocab`
  - counts all answers across training set and keeps the 1,000 most common
  - maps each answer string to an integer index
- `get_image_transform`
  - applies random cropping and flipping to training data
  - applies center cropping for validation data
  - resizes to 224x224 for ViT

This prevents I/O bottlenecks and reduces GPU starvation

VQA Dataset Class

```python
class VQADataset(Dataset):
    """PyTorch dataset for VQA v2.0 with HDF5 image loading and soft targets.
    # ... (init and h5 file initialization)

    # --- Soft target construction ---
    target = torch.zeros(num_answers, dtype=torch.float)
    for a in ann["answers"]:
        ans_text = a["answer"]
        if ans_text in self.answer_to_idx:
            target[self.answer_to_idx[ans_text]] += 1.0

    if target.sum() == 0:
        continue

    target /= 10.0  # normalize: count -> fraction of annotators
    # ... (returns image, input_ids, attention_mask, sample["answer_target"])
```

- builds VQA targets based on human annotators
- questions outside of the top-1000 vocabulary (NUM_ANSWERS) are skipped
- This class is instantiated using the .json files in training data

Model Definitions - ImageEncoder & Text

```python
class ImageEncoder(nn.Module):
    """ViT-B/16 image encoder. Output: (B, 197, embed_dim)."""

    VIT_HIDDEN_DIM = 768

    def __init__(self, embed_dim=512, freeze=True):
        super().__init__()
        vit = vit_b_16(weights=ViT_B_16_Weights.IMAGENET1K_V1)
        self.conv_proj = vit.conv_proj
        self.class_token = vit.class_token
        self.encoder = vit.encoder
        # Only this projection layer is trainable — all ViT weights above are frozen.
        # This maps from ViT's native 768-dim to our shared embedding space (512-dim).
        self.projection = nn.Linear(self.VIT_HIDDEN_DIM, embed_dim)

        if freeze:
            for param in self.conv_proj.parameters():
                param.requires_grad = False
            self.class_token.requires_grad = False
            for param in self.encoder.parameters():
                param.requires_grad = False

    def forward(self, images):
        # conv_proj splits 224x224 image into 16x16 patches -> 14x14 = 196 patch embeddings
        x = self.conv_proj(images)                          # (B, 768, 14, 14)
        x = x.flatten(2).transpose(1, 2)                    # (B, 196, 768)
        # Prepend CLS token: a learnable vector that aggregates whole-image information
        # through self-attention. After encoding, CLS (index 0) serves as the image summary.
        cls = self.class_token.expand(x.shape[0], -1, -1)  # (B, 1, 768)
        x = torch.cat([cls, x], dim=1)                      # (B, 197, 768)
        x = self.encoder(x)                                 # (B, 197, 768)
        return self.projection(x)                           # (B, 197, embed_dim)


class TextEncoder(nn.Module):
    """RoBERTa-base text encoder. Output: (B, seq_len, embed_dim)."""

    ROBERTA_HIDDEN_DIM = 768

    def __init__(self, embed_dim=512, freeze=True):
        super().__init__()
        self.roberta = RobertaModel.from_pretrained("roberta-base")
        # Same idea as ImageEncoder: only this projection is trainable
        self.projection = nn.Linear(self.ROBERTA_HIDDEN_DIM, embed_dim)

        if freeze:
            for param in self.roberta.parameters():
                param.requires_grad = False

    def forward(self, input_ids, attention_mask):
        outputs = self.roberta(input_ids=input_ids, attention_mask=attention_mask)
        return self.projection(outputs.last_hidden_state)  # (B, seq_len, embed_dim)
```

- reduces output dimensions from 768x768 to 512x512 (used in our images)
- pre-trained weights are not updated during VQA training

Cross Attention Block

```python
class CrossAttentionBlock(nn.Module):
    # Pre-norm cross-attention with residual connections + FFN
    def forward(self, query, key_value, key_padding_mask=None):
        q = self.norm_q(query)
        kv = self.norm_kv(key_value)
        attended, attn_weights = self.cross_attn(q, kv, kv, ...)
        query = query + attended              # residual
        query = query + self.ff(self.norm_ff(query))   # FFN + residual
        return query, attn_weights

class AsymmetricCrossModalFusion(nn.Module):
    # TWO separate, independent CrossAttentionBlocks
    def forward(self, image_features, text_features, text_padding_mask=None):
        # Step 1: image queries attend to text
        img_attended, attn_i2t = self.cross_attn_img_to_txt(query=image_features,
key_value=text_features)
        # Step 2: text queries attend to ALREADY-GROUNDED image (not raw image)
        txt_attended, attn_t2i = self.cross_attn_txt_to_img(query=text_features, key_value=img_attended)
        return img_attended, txt_attended, attn_i2t, attn_t2i
```

```python
class SymmetricCrossModalFusion(nn.Module):
    """ONE SHARED cross-attention block used in both directions (baseline).
    Both directions share the same weights and operate on RAW encoder outputs
    independently — no sequential information flow between them."""

    def __init__(self, embed_dim, num_heads=8, dropout=0.1):
        super().__init__()
        self.shared_cross_attn = CrossAttentionBlock(embed_dim, num_heads, dropout)

    def forward(self, image_features, text_features, text_padding_mask=None):
        # Same shared weights for both directions — a compromise, since the
        # optimal attention pattern for image->text differs from text->image
        img_attended, attn_i2t = self.shared_cross_attn(
            query=image_features, key_value=text_features,
            key_padding_mask=text_padding_mask)
        txt_attended, attn_t2i = self.shared_cross_attn(
            query=text_features, key_value=image_features)
        return img_attended, txt_attended, attn_i2t, attn_t2i
```

- creates two independent blocks with separately learned weights
- fuses them using the saved `query` and `key_value`
  - text → image
  - image → text

- `class SymmetricCrossModalFusion` uses a shared block for both directions